Základy počítačové grafiky
Přednáška 5

Martin Němec

VŠB-TU Ostrava

2026

VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY | KATEDRA INFORMATIKY

Opakování

Aktuálně:

- Vykreslování objektů (VAO+VBO).
- Modelové transformace.
- Pohledová transformace.
- Projekční transformace.

Je něco, co byste chtěli zopakovat?

Objektově orientované programování

Co si pamatujete z návrhu kódu?

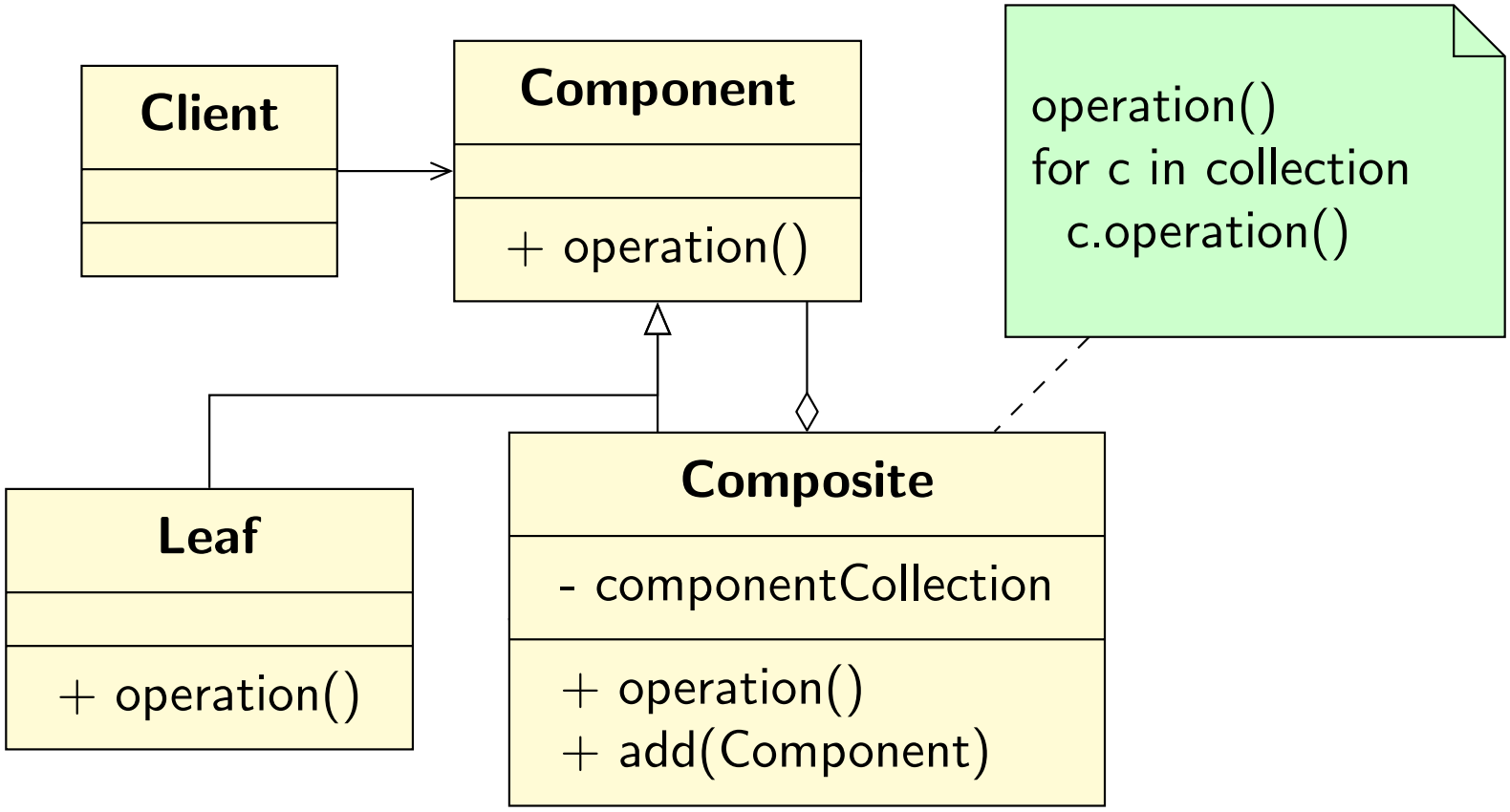
- Jasně definujte povinnosti/zodpovědnost.
- Každá třída by měla mít pouze jednu odpovědnost.
  To zjednodušuje údržbu a rozšiřování kódu.
- Pozor na "božské" třídy, které všechno ví a všechno umí.
- Třídy by měly být navrženy tak, aby bylo snadné přidat nové funkce (rozšiřovat je) bez nutnosti měnit stávající kód.
- Důležitá je abstrakce kódu.

Návrh tříd není jednorázový proces. Během vývoje bývá potřeba některé třídy a jejich vazby přidat, upravit, rozdělit apod.

Refaktoring je klíčová součást vývoje OOP, používáme nástroje jako UML diagramy.

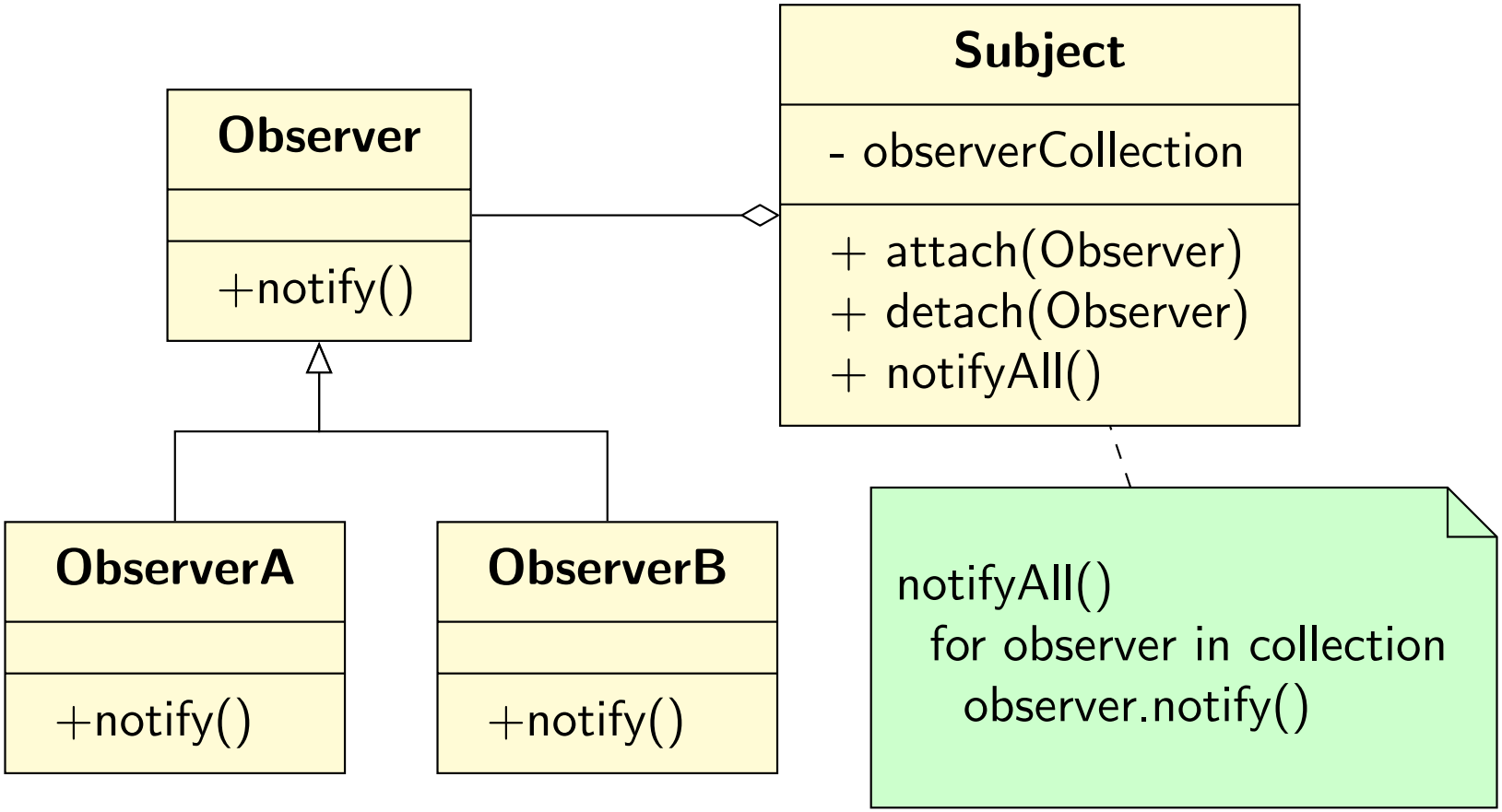
Composite Pattern

Přistupujeme stejným způsobem k jednoduchým i složeným (kompozitním) objektům (souborový systém).



Observer Pattern

Umožňuje jednomu objektu (Subject) informovat ostatní objekty (Observers) o změnách svého stavu, aniž by je přímo znal.



Kulová plocha

```
const float sphere[17280] = {
-0.831,-0.555,0.000,-0.833,-0.552,0.000,
-0.923,-0.382,0.000,-0.924,-0.380,0.000,
-0.815,-0.555,-0.162,-0.817,-0.552,-0.162,

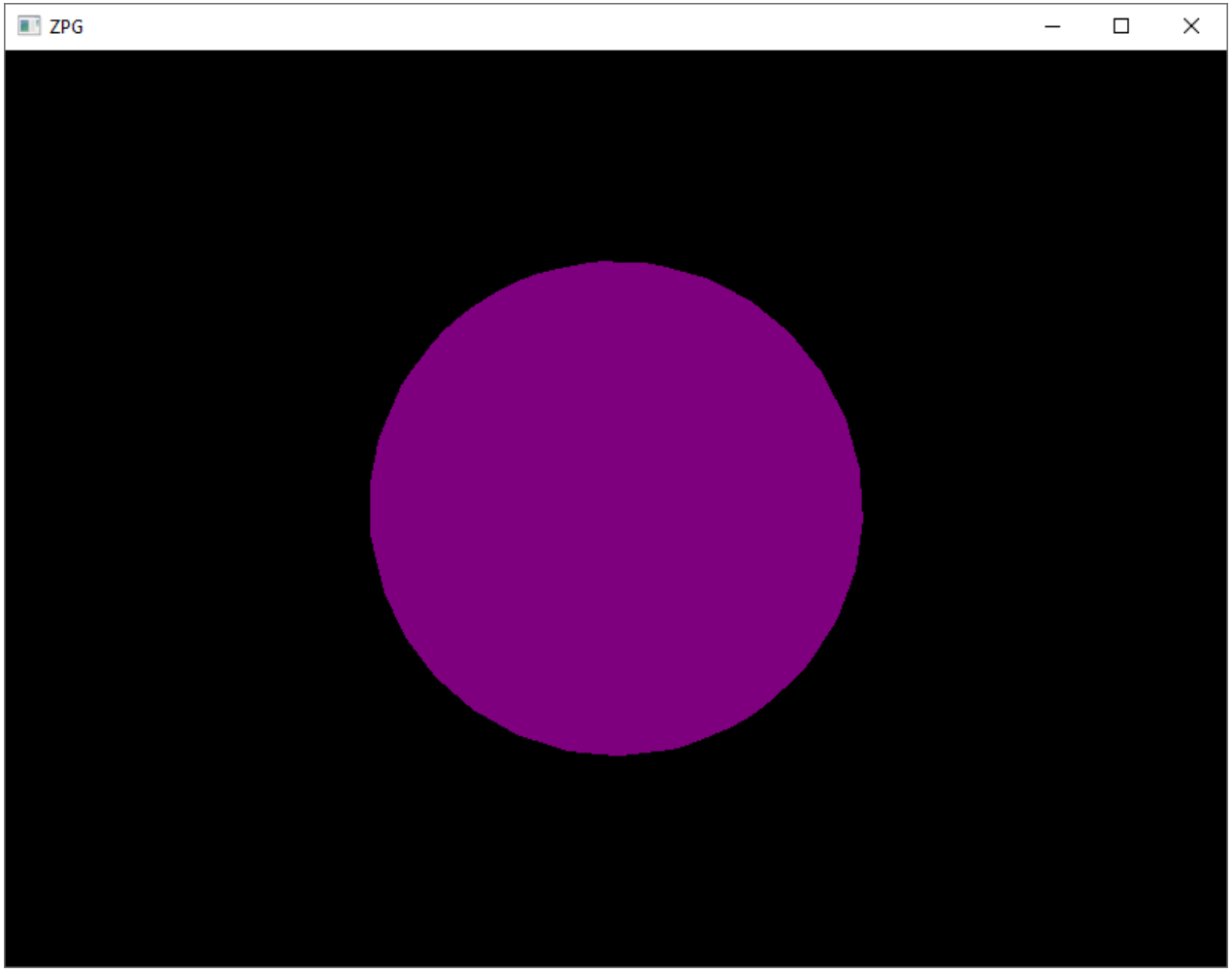
-0.375,-0.923,0.074,-0.380,-0.921,0.075,
-0.555,-0.831,0.000,-0.559,-0.828,0.000};
```




```
glVertexAttribPointer(
  0,3,GL_FLOAT,GL_FALSE,6*sizeof(float),(GLvoid*)0);
glVertexAttribPointer(
  1,3,GL_FLOAT,GL_FALSE,6*sizeof(float),
     (GLvoid*)(3*sizeof(GLfloat)));
```

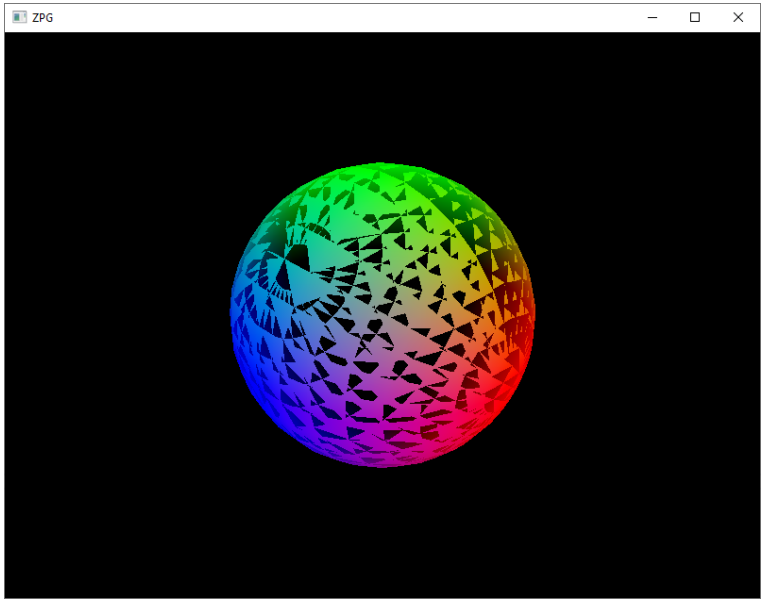
Kulová plocha

Přidání tělesa (kulové plochy) do scény. Jsme spokojení?



Kulová plocha

Problém se "špatným" vykreslováním trojúhelníků vyřešíte zapnutím z-bufferu. Viditelnosti se budeme věnovat na některé z dalších přednášek.



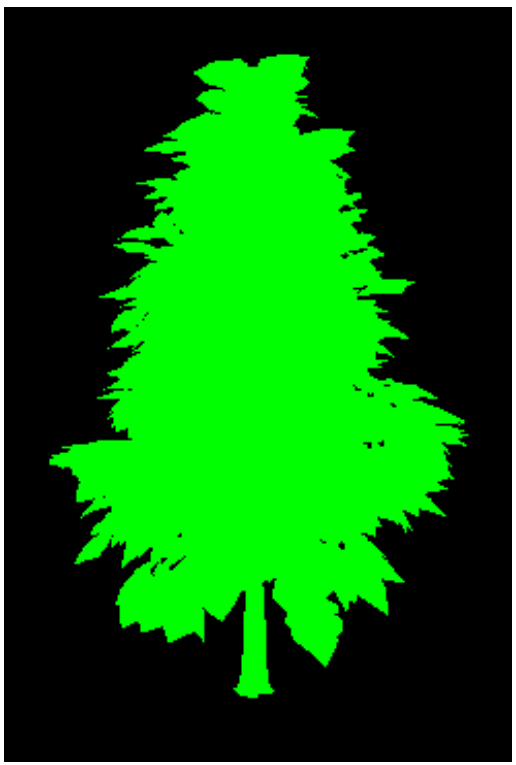

```
glEnable(GL_DEPTH_TEST);
while (!glfwWindowShouldClose(window)) {
  glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

Vykreslení stromu

Postupně jsme upravovali výsledný vzhled a používali různé techniky (naposledy lineární mapování na interval).

$$x' = a + \frac{x - x_{\min}}{x_{\max} - x_{\min}}(b - a)$$







Osvětlovací modely

Dosud jsme vzhled objektů tvořili pomocí geometrie a barev.

Nyní využijeme **osvětlovací model**, který bude určovat výsledný vzhled povrchu objektu na základě působení světla.

- Světlo ovlivňuje výslednou barvu a jas povrchu.
- Výsledný vzhled závisí na poloze a vlastnostech světla.
- Důležitou roli hrají také vlastnosti materiálu a orientace povrchu.

Přístupy k vykreslování osvětlení:

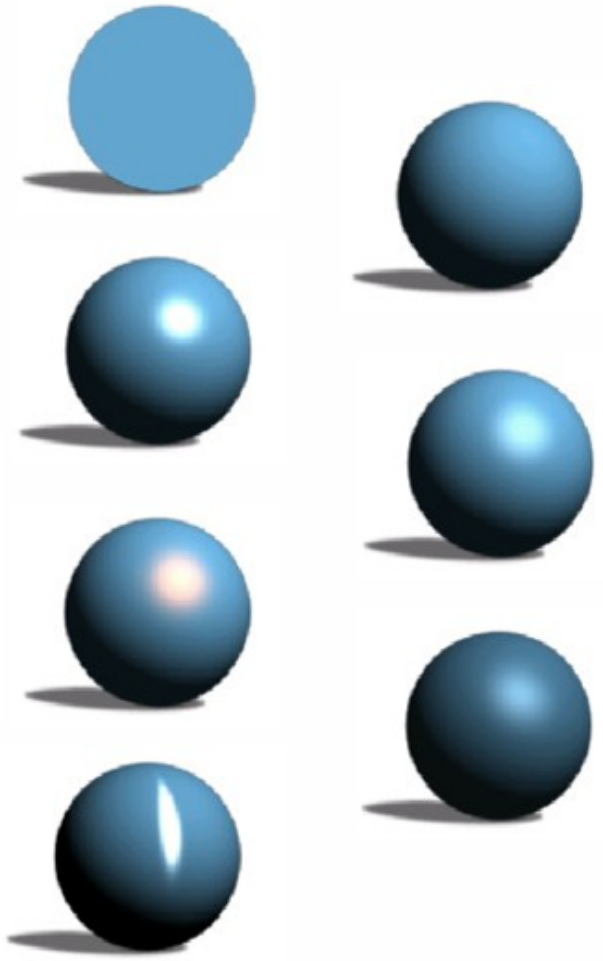
- **Real-time rendering** – výpočetně efektivní modely pro interaktivní aplikace a počítačové hry.
- **Fotorealistický rendering** – fyzikálně založené metody pro co nejvěrnější simulaci šíření světla (předmět Počítačová grafika).

Osvětlovací modely

Výpočet barvy na povrchu těles

- Constant
- Lambert
- Phong
- Blinn
- Cook-Torrence
- Strauss
- Anisotropic
- a další.

(kvalita vs. rychlost)



Konstantní shader

Ambientní složka zabraňuje tomu, aby povrchy odvrácené od světelných zdrojů byly zcela černé.

- pouze konstantní barva
- ignorují se normály
- stejná orientace ploch
- stejná vzdálenost



```
outColor = vec4( 0.385, 0.647, 0.812, 1.0 );
```

Lambertův shader

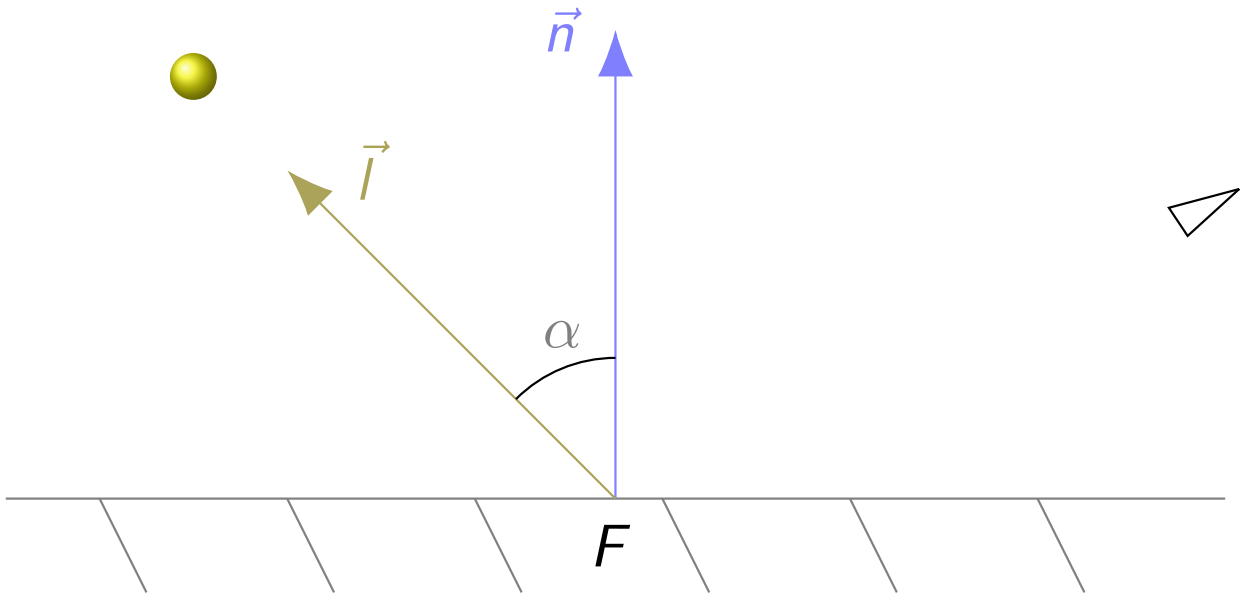
- ambientní a difúzní složka
- neobsahuje zrcadlovou složku
- matný povrch



```
I=max(dot(lightVector,worldNor), 0.0);
```

Lambertův shader

Intenzita světla je větší, čím menší je úhel mezi směrem ke světlu a normálou povrchu.




```
Id=max(dot(lightVector,worldNor), 0.0);
```

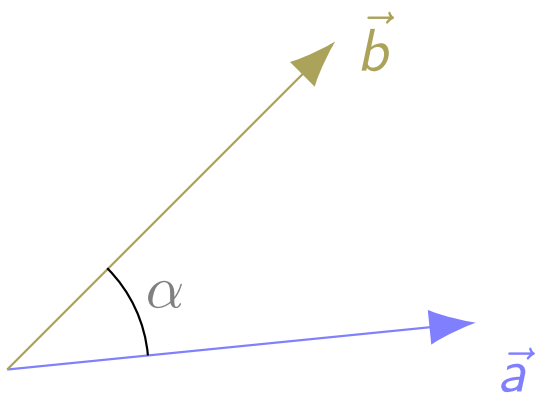
Skalární součin

Definuje se mezi dvěma vektory a zachycuje vztah mezi velikostí těchto vektorů a jejich úhlem.

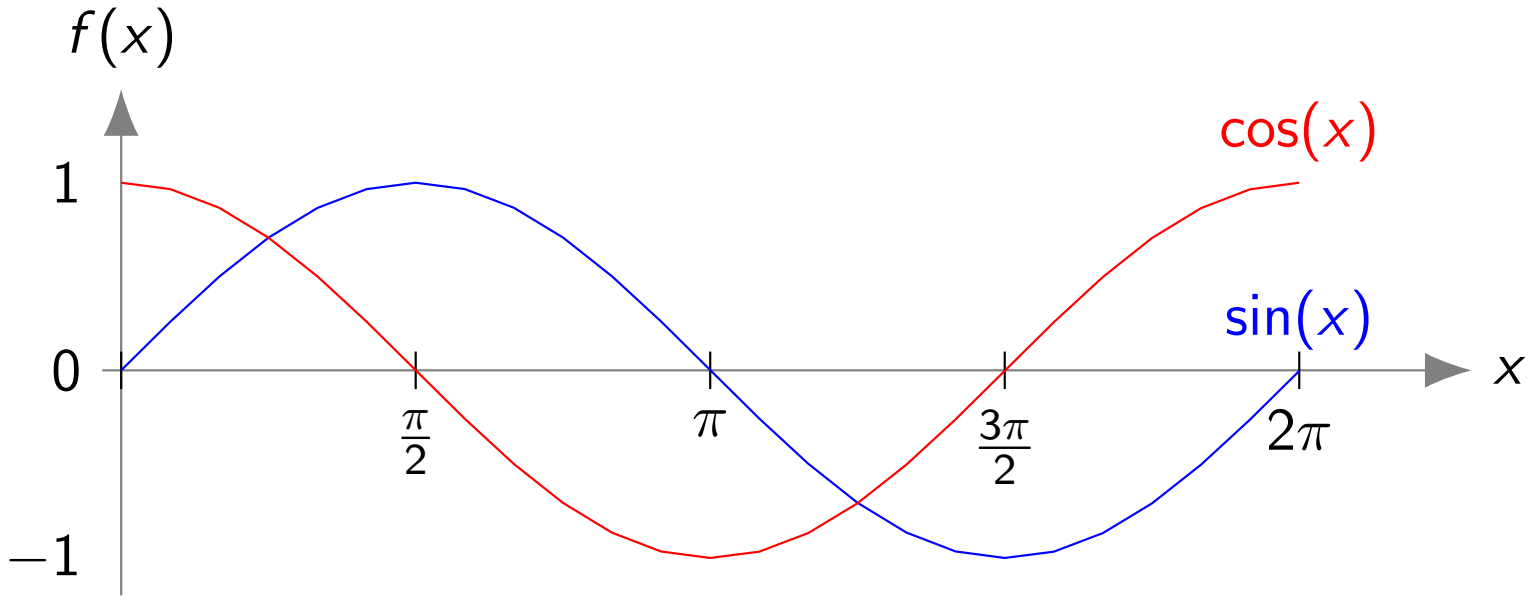
$$\langle \vec{a}, \vec{b} \rangle = \|\vec{a}\| \|\vec{b}\| \cos \alpha$$

$$\langle \vec{a}, \vec{b} \rangle = a_1 b_1 + a_2 b_2 + a_3 b_3$$

$$\langle \vec{a}, \vec{b} \rangle = dot(\vec{a}, \vec{b})$$



Goniometrické funkce sinus a kosinus



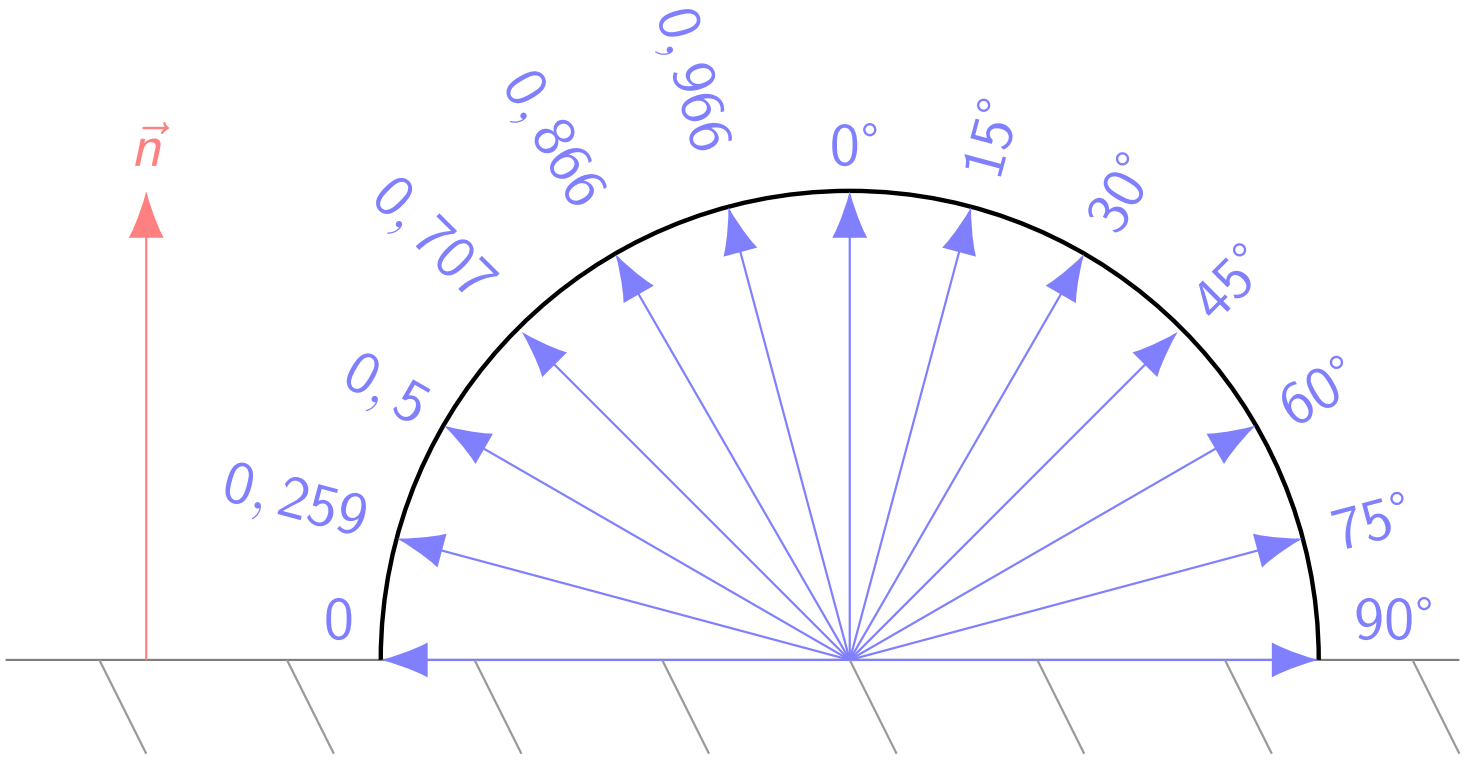
Funkce $\sin(x)$ a $\cos(x)$ jsou základní periodické funkce, které vycházejí z pravoúhlého trojúhelníku a jednotkové kružnice.

Kosinus je posunutý oproti sinus o $\pi/2$: $\cos(x) = \sin(x + \pi/2)$.

Definiční obor: $D(\sin) = D(\cos) = \mathbb{R}$
Obor hodnot: $H(\sin) = H(\cos) = \langle -1, 1 \rangle$

Lambertův shader

Proč kosinus? Význam $(max(dot\left(\hat{n}, \hat{l}\right), 0))$.



Vertex shader

```
#version 400
layout(location = 0) in vec3 vp; //vertex position
layout(location = 1) in vec3 vn; //vertex normal

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;
out vec3 worldPos;
out vec3 worldNorm;

void main(void)
{
  //jedna se o spravny prevod?
  worldPos = vec3(model * vec4(vp,1.0));
  worldNorm = vn; //problem
  gl_Position = projection * view * model * vec4(vp,1.0);
}
```

Lambert - Fragment shader

```
#version 460 core
in vec3 worldPos;
in vec3 worldNor;
out vec4 fragColor;

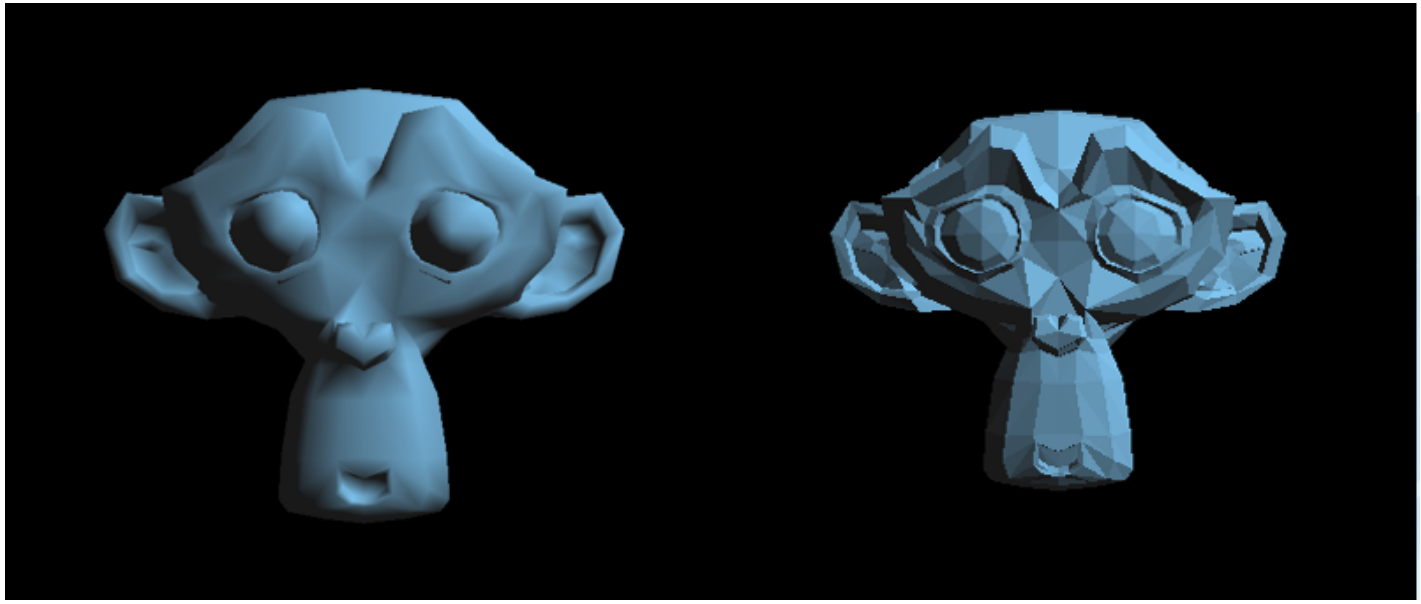
void main(void)
{
  vec3 lightPosition= vec3(0.0,10.0,0.0);
  vec3 lightDir = ?;

  float diff=
    max(dot(normalize(lightDir),normalize(worldNor))0.0);
    vec3 ambient = vec3(0.1, 0.1, 0.1);
    vec3 objectColor = vec3(0.385, 0.647, 0.812);
    vec3 diffuse = diff * objectColor;
    fragColor = vec4(ambient + diffuse, 1.0)

}
```

Lambertův shader

Rozdíl v nastavení Smooth vs. Flat?



Phongův shader

Phongův osvětlovací model navrhl v roce 1973 v rámci své disertační práce Bui Tuong Phong.

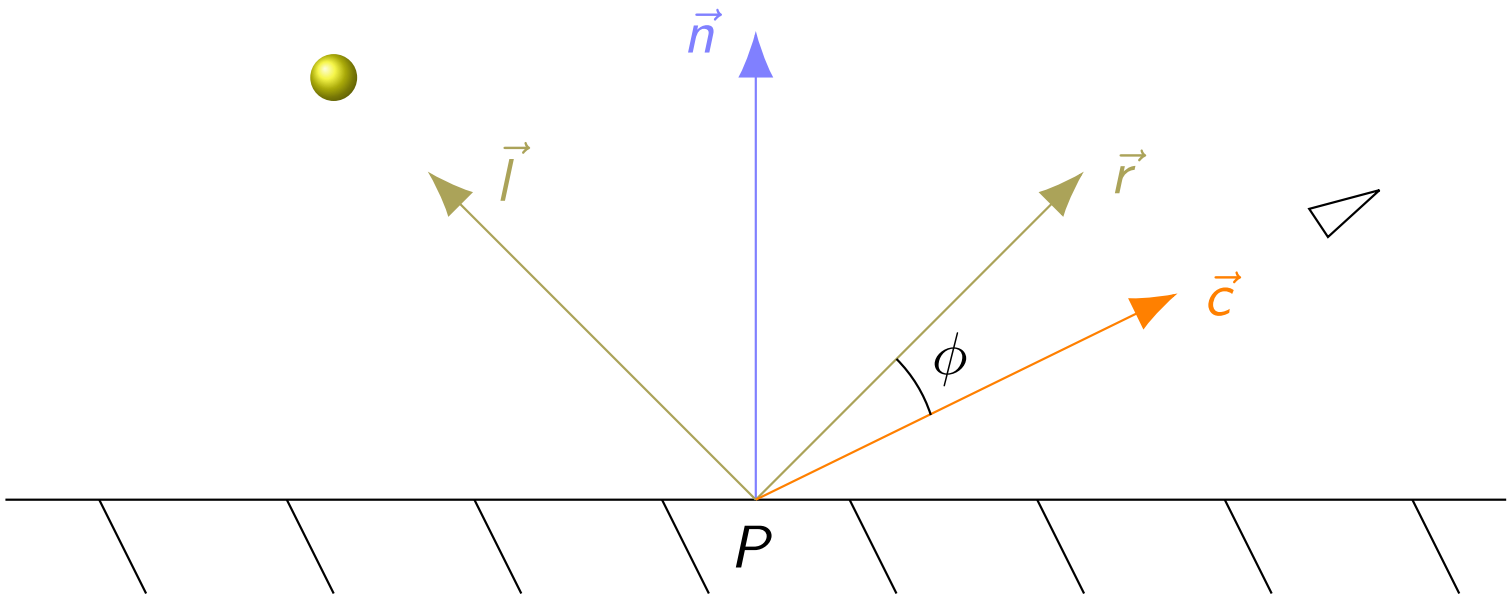
- Ambientní, difúzní a zrcadlová složka.
- Vytváří na povrchu odlesky.
- Empirický model, není založený na fyzice.



```
I=Ia+Id+Is;
```

Phongův shader

Zrcadlová složka je závislá na úhlu mezi odraženým vektorem $\vec{r}$ a směrem ke kameře $\vec{c}$.




```
Is=max(dot(reflect(light, normal),camera), 0.0);
```

Phong - Fragment shader

```
void main(void)
{
  ...
  vec3 reflectDir = reflect(-lightDir, worldNor);
  float shininess = 32.0; //urcuje ostrost odlesku
  spec= pow(max(dot(viewDir,reflectDir),0.0),shininess);
  vec3 lightColor=vec3(1.0,1.0,1.0);
  vec3 specular=specularStrength*spec*lightColor;;

  // I=Ia+Id+Is
  fragColor = vec4(ambient + diffuse + specular, 1.0);
}
```

Phongův osvětlovací model

Ambientní složka

$$I_{ambient} = I_a \cdot r_a$$

Difúzní složka

$$I_{diffuse} = I_d \cdot r_d \max(0, \vec{n} \cdot \vec{l})$$

Zrcadlová složka

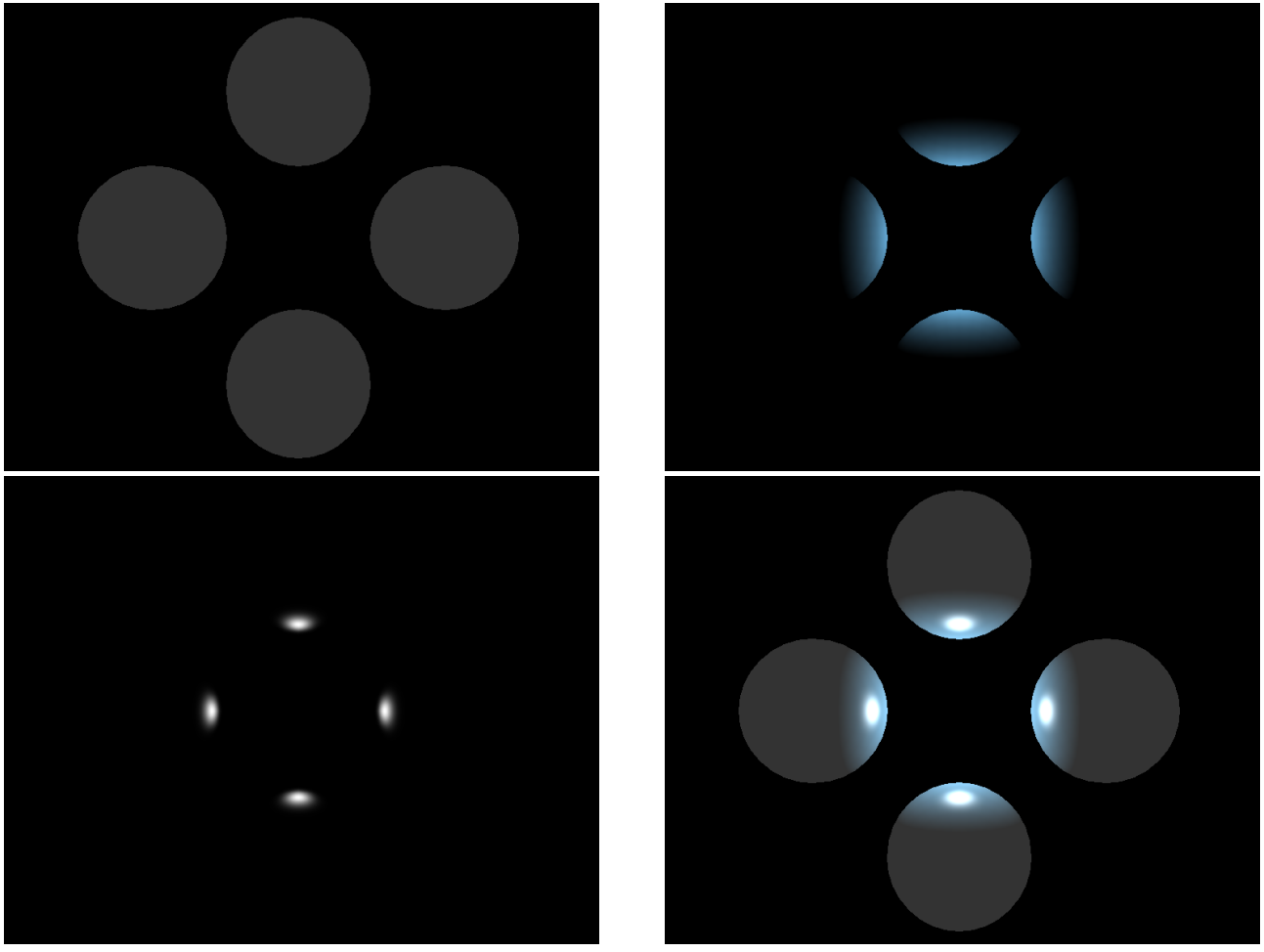
$$I_{specular} = I_s \cdot r_s \max(0, \vec{r} \cdot \vec{c})^s$$

Obecně lze osvětlovací model zapsat takto:

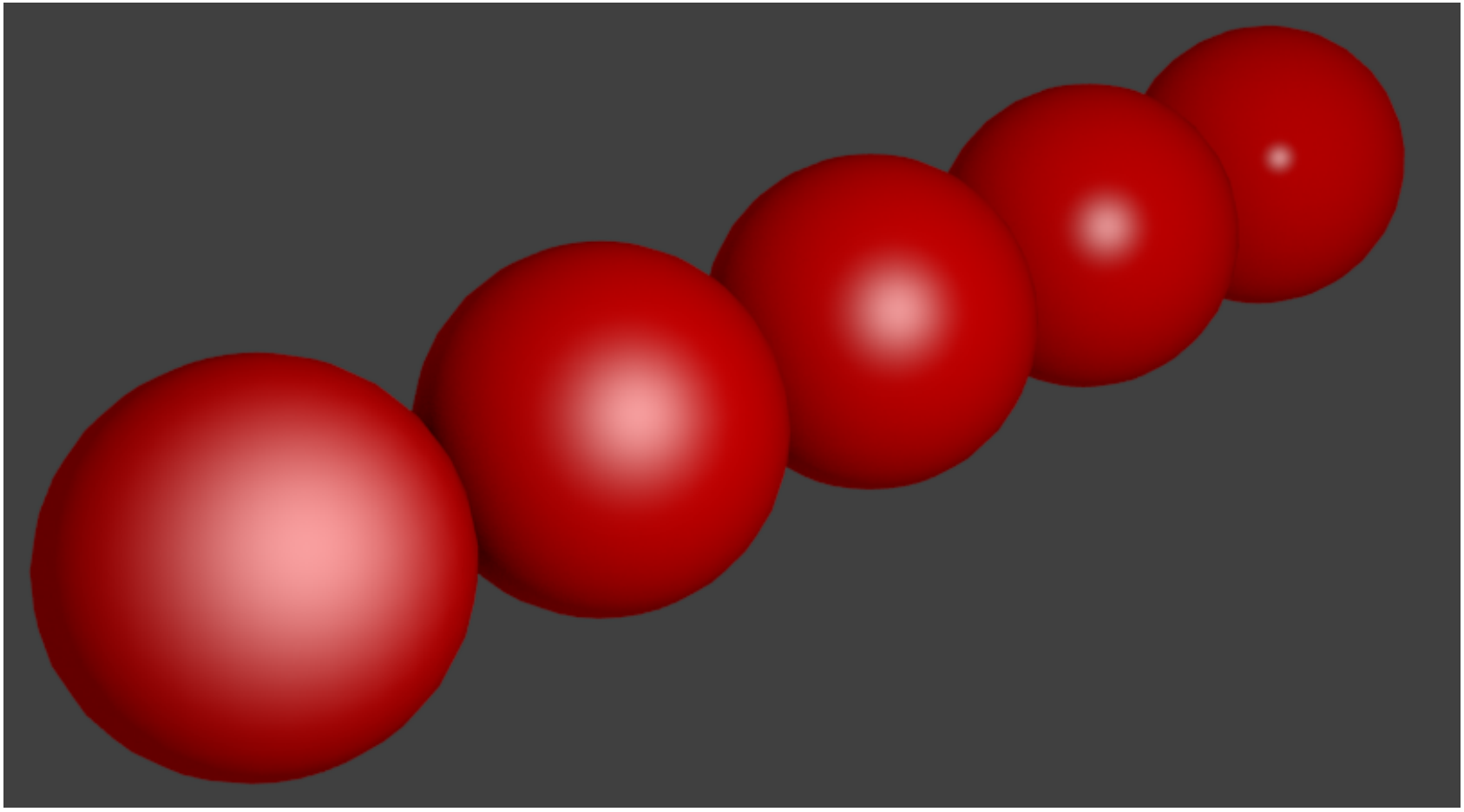
$$I = I_a r_a + \sum_{i=0}^{m} \left(I_{d,i} r_d \cos \alpha_i + I_{s,i} r_s \cos^h \phi_i \right)$$

$$I = I_a r_a + \sum_{i=0}^{m} \left(I_{d,i} r_d (\hat{l} \cdot \hat{n}) + I_{s,i} r_s (\hat{r} \cdot \hat{v})^h \right)$$

Phongův osvětlovací model

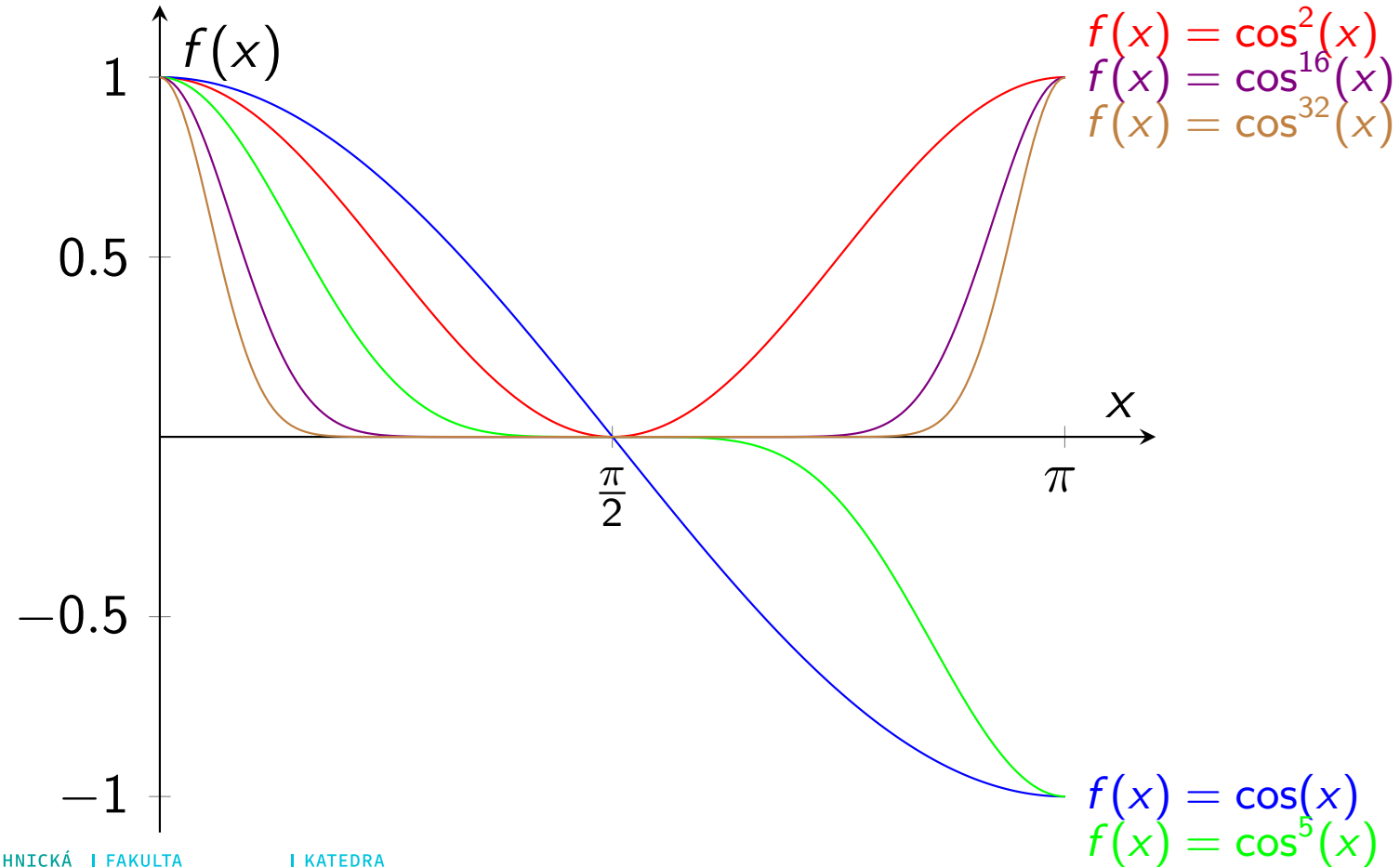


Shininess constant



Shininess constant

Ukázka změny průběhu kosinové funkce při různých hodnotách mocniny, která se používá u zrcadlové složky $(max(\cos\theta, 0))^h$.



Použití funkcí v GLSL

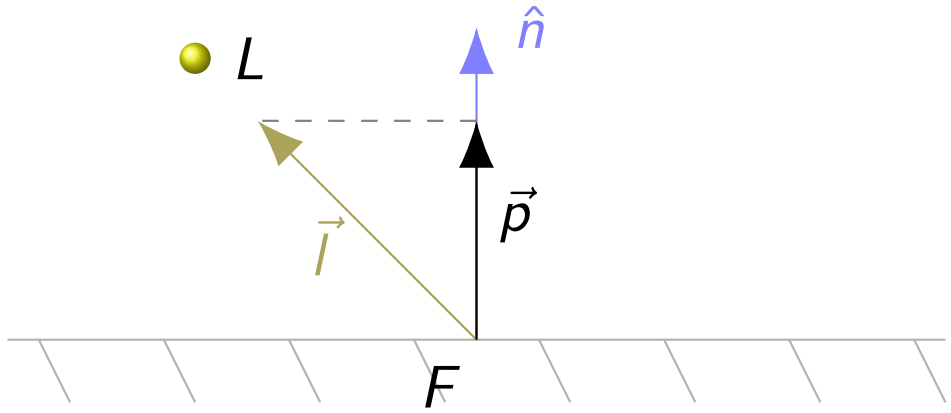
V GLSL můžeme pro větší přehlednost kódu využívat vlastní funkce.

```
// Calculate diffuse light intensity
float calculateDiffuse(vec3 lightDir, vec3 normal)
{
    float dotLN = dot(normalize(lightDir),
                      normalize(normal));
    return max(dotLN, 0.0);
}

void main()
{
    ...
    // Calculate diffuse component
    float diff = calculateDiffuse(lightDir, normal);
    vec3 diffuse = diff * color;
    ...
}
```

Reflection Vector

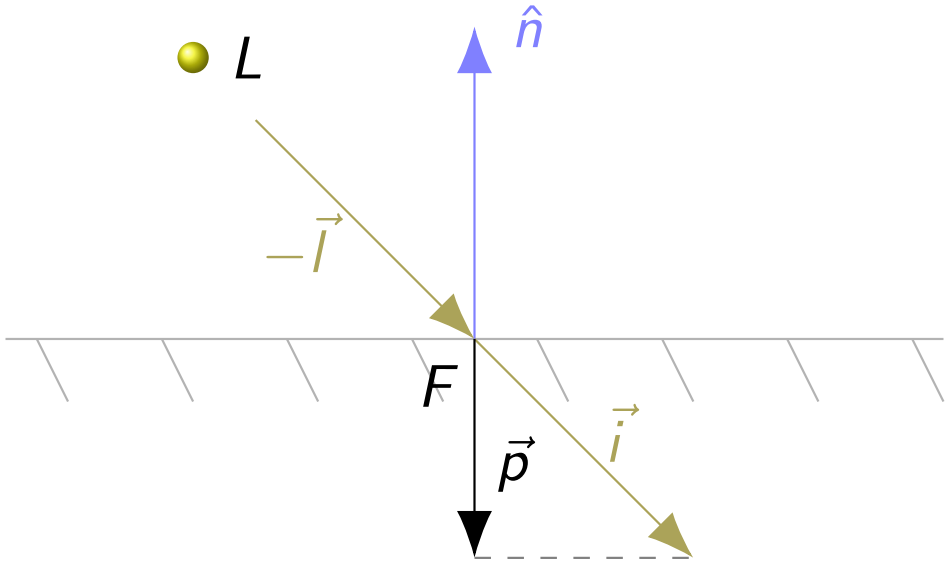
Nejprve si vypočteme vektor $\vec{p}$ jako projekci vektoru $\vec{l}$ na normalizovanou (jednotkovou) normálu $\vec{n}$ (značíme $\hat{n}$).



$$\vec{p} = (\hat{n} \cdot \vec{l})\hat{n}\,.$$

Projekce vektoru na normálu

V GLSL funkci reflect se pro výpočet používá vektor dopadu světla, takže opačný vektor než ke světlu $\vec{l}$, mějme tedy $\vec{i} = -\vec{l}$.



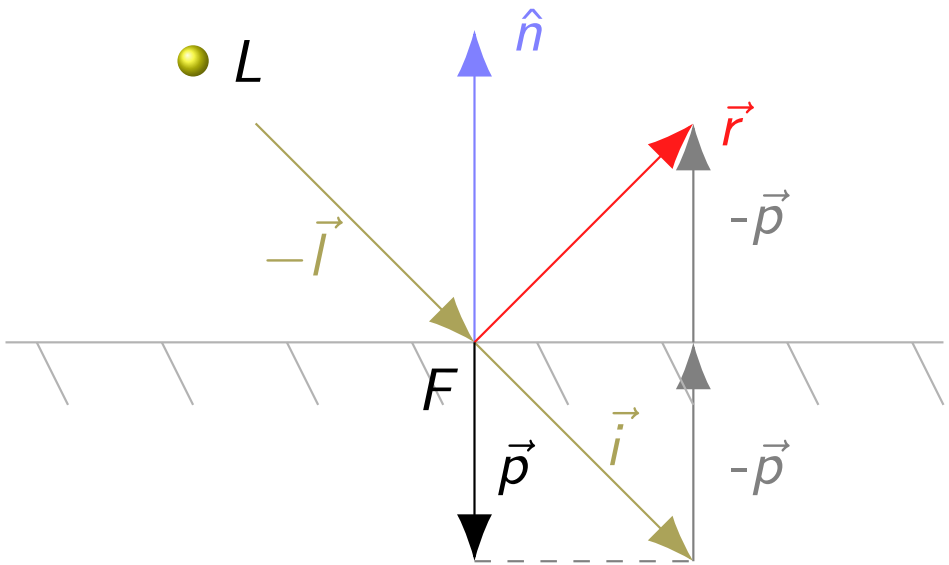
Výsledný vektor $\vec{p}$ je nyní zadaný projekcí vektoru $\vec{i}$ na normálu $\hat{n}$.

$$\vec{p} = (\hat{n} \cdot \vec{i})\hat{n}\,.$$

Odražený paprsek

Odražený paprsek $\vec{r}$ lze v případě normalizované normály získat jako (pozor na správnou orientaci světla $-\vec{l} = \vec{i}$):

$$\vec{r} = \vec{i} - 2\vec{p} = \vec{i} - 2(\hat{n} \cdot \vec{i})\hat{n}\,.$$



Funkce reflect v GLSL

V GLSL má funkce reflect tvar: **vec3 reflect(vec3 I, vec3 N);**

```
void main(void)
{
    // from the fragment to the light
    vec3 L = normalize(lightPos - fragPos);
    vec3 R = reflect(-L, N);
}
```

$$\vec{r} = \vec{i} - 2\vec{p} = \vec{i} - 2\left(\hat{n} \cdot \vec{i}\right)\hat{n}\,.$$

Pokud bude vektor $\vec{i}$ normalizovaný, tak bude normalizovaný i odražený vektor $\vec{r}$.

$$\hat{r} = \hat{i} - 2\vec{p} = \hat{i} - 2\left(\hat{n} \cdot \hat{i}\right)\hat{n}\,.$$

Příklad na reflect

Mějme světlo na pozici $L = [-1, 1, 0]$, fragment $F = [0, 0, 0]$ je ve středu souřadné soustavy a normála $\vec{n} = (0, 1, 0)$.

- Normalizujeme vektory

$$\hat{n} = (0, 1, 0)\,, \vec{i} = (F - L) = (1, -1, 0)\,, \hat{i} = \left(\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}, 0\right)\,.$$

- Skalární součin

$$dot\left(\hat{n}, \hat{i}\right) = -\frac{1}{\sqrt{2}}\,.$$

- Vypočteme odražený vektor (reflect)

$$\vec{r} = \vec{i} - 2 \cdot dot\left(\hat{n}, \vec{i}\right) \hat{n}\,.$$

$$\hat{r} = \hat{i} - 2 \cdot dot\left(\hat{n}, \hat{i}\right) \hat{n}\,.$$

Příklad na reflect

Výpočet odraženého vektoru:

$$\hat{r} = \hat{i} - 2 \cdot dot\left(\hat{n}, \hat{i}\right) \hat{n}\,,$$

$$\hat{r} = \left(\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}, 0\right) - 2 \cdot \left(-\frac{1}{\sqrt{2}}\right)(0, 1, 0)\,,$$

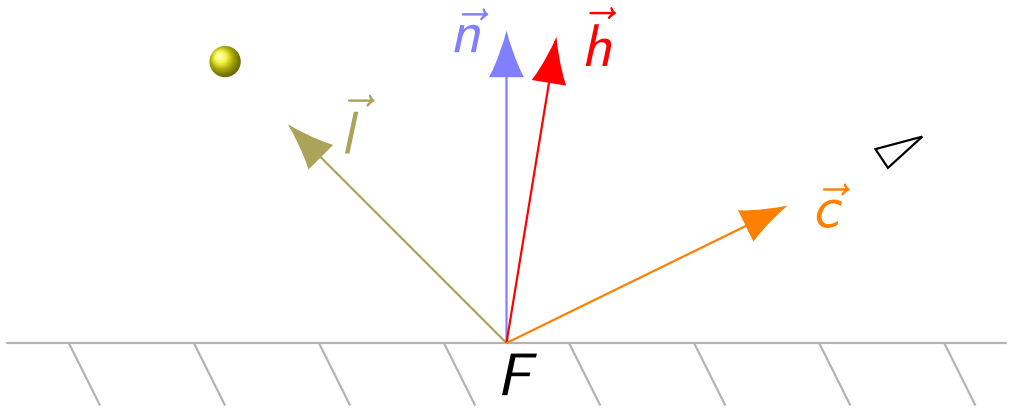
$$\hat{r} = \left(\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}, 0\right) + \left(0, \frac{2}{\sqrt{2}}, 0\right)\,,$$

$$\hat{r} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0\right)\,.$$

Odražený vektor má souřadnice $\hat{r} = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0\right)$.

Blinn–Phong reflection model

James F. Blinn (americký informatik a počítačový grafik) upravil v roce 1977 osvětlovací model a nahradil náročnější výpočet reflect() pomocí halfway vektoru (používá se jen jeden skalární součin).



$$\hat{h} = \frac{\vec{l} + \vec{c}}{|\vec{l} + \vec{c}|}$$

```
vec3 halfwayDir = normalize(lightDir + viewDir);
```

$$I_{specular} = I_s \cdot r_s \max\left(0, \langle \hat{n}, \hat{h} \rangle\right)^s$$

Light attenuation

Obecně se vychází z toho, že intenzita klesá s druhou mocninou vzdálenosti od světelného zdroje.

$$f_{att}(d) = \frac{1}{k_c + k_l d + k_q d^2}$$

Hodnoty nejsou předem dané, upravujeme je podle požadavků na výslednou scénu.

```
glLightfv(GL_LIGHT0, GL_CONSTANT_ATTENUATION, 2.0);
glLightfv(GL_LIGHT0, GL_LINEAR_ATTENUATION, 2.0);
glLightfv(GL_LIGHT0, GL_QUADRATIC_ATTENUATION, 2.0);
```

Konstantní, Gouraudovo a Phongovo stínování

Stínování popisuje to, jak se nanáší barva na jednotlivé plochy

- Konstantní - jednoduchá a rychlá metoda, používaná pro náhled, vypočítá barvu (osvětlovací model) v jednom bodě a použije ji pro celý polygon (velmi rychlé).
- Gouraudovo stínování (interpolace barvy) - vypočítá barvu ve všech vrcholech polygonu a ta se interpoluje po celém polygonu.
- Phongovo stínování (interpolace normály) - barva (osvětlovací model) se počítá v každém fragmentu samostatně (nejhezčí, časově nejnáročnější).

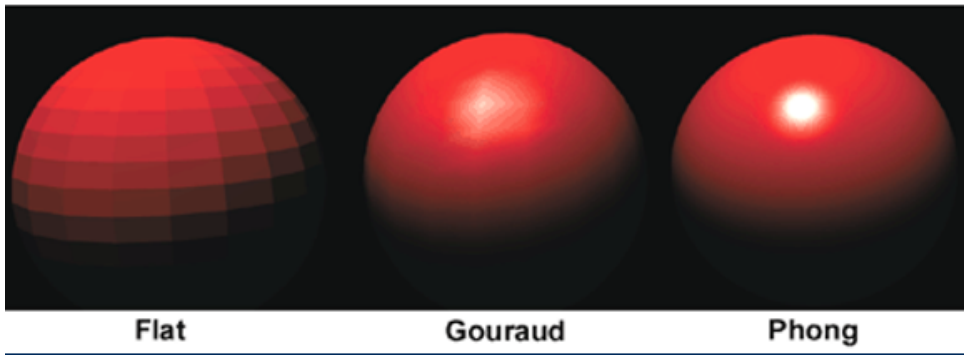
Jak by to fungovalo v našem vertex a fragment shaderu?

Konstantní, Gouraudovo a Phongovo stínování

Stínování popisuje to, jak se nanáší barva na jednotlivé plochy.



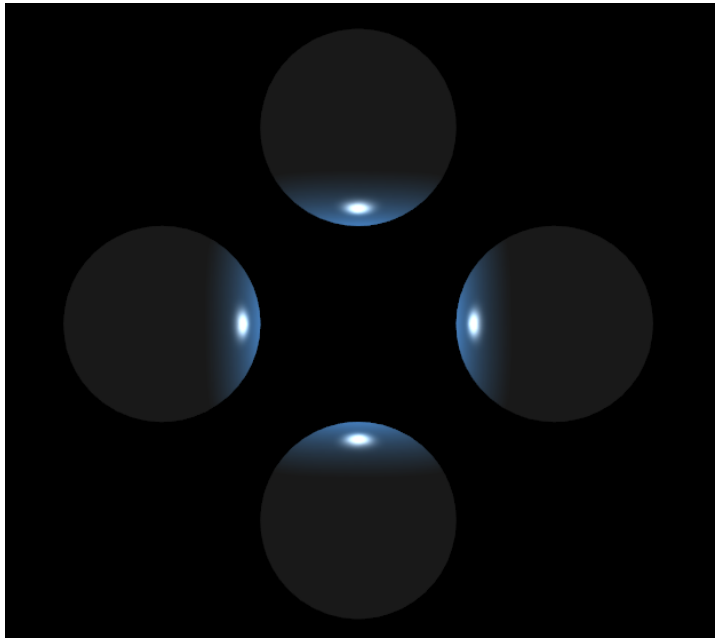
From Computer Desktop Encyclopedia
Reproduced with permission.
© 2001 Intergraph Computer Systems



Příprava scény a testování

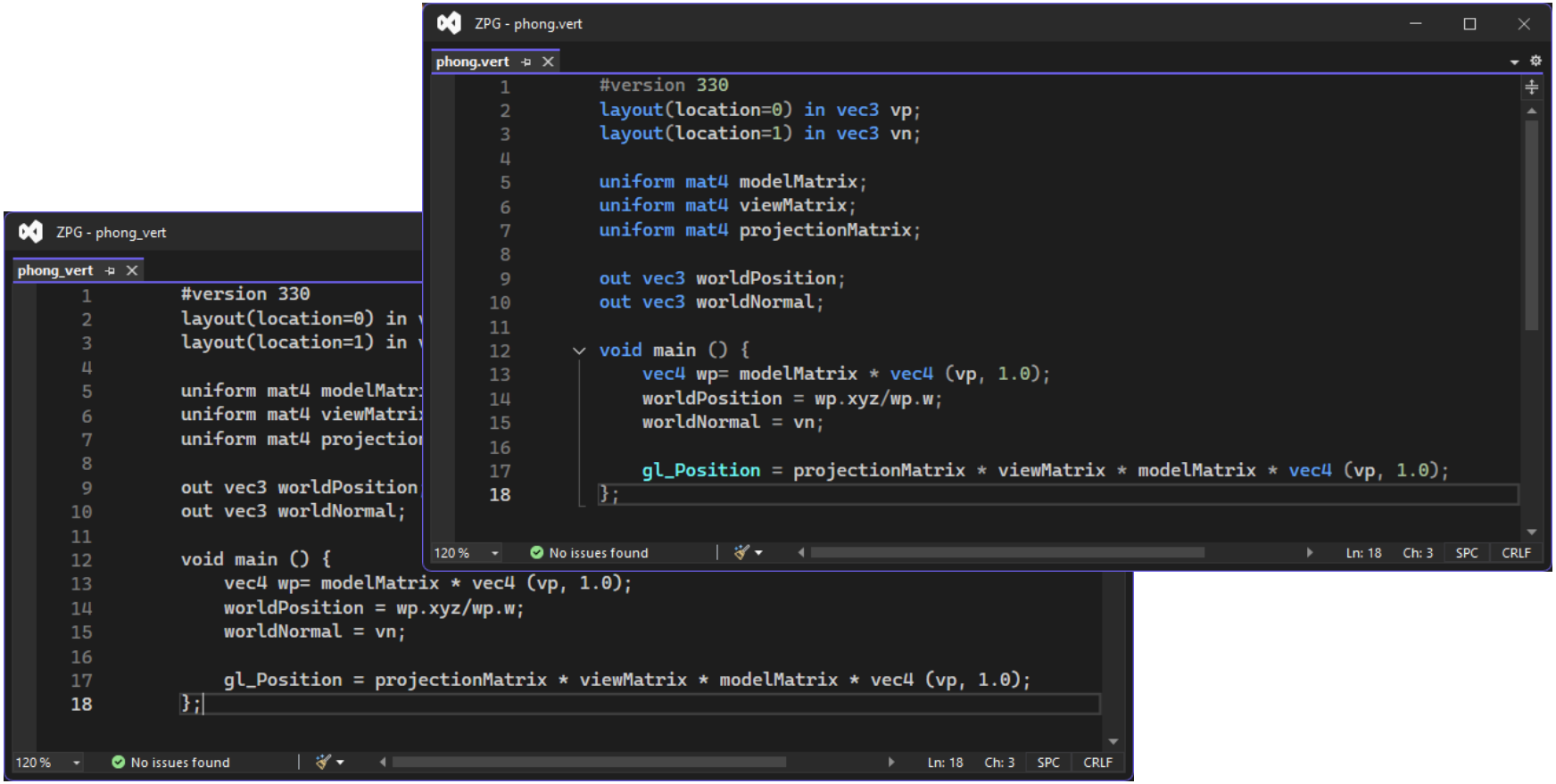
Světlo v [0,0,0], čtyři kulové plochy souměrně rozmístěny a kamera před objekty.
Kontrolujeme všechny složky (ambientní, zrcadlovou a difúzní).

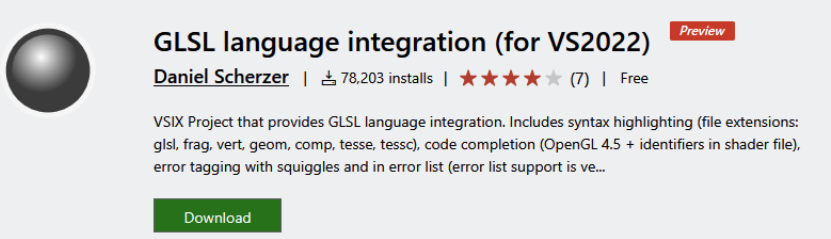
Problém s transformaci normály (normálová matice) - příště na přednášce.



GLSL language integration

GLSL language integration (for VS2022 and VS2026) Daniel Scherzer





Závěr

Dotazy?